Clustering

K-means clustering

K-means clustering is an unsupervised learning algorithm that classifies a data point based on the majority of its neighbors.

The algorithm is as follows:

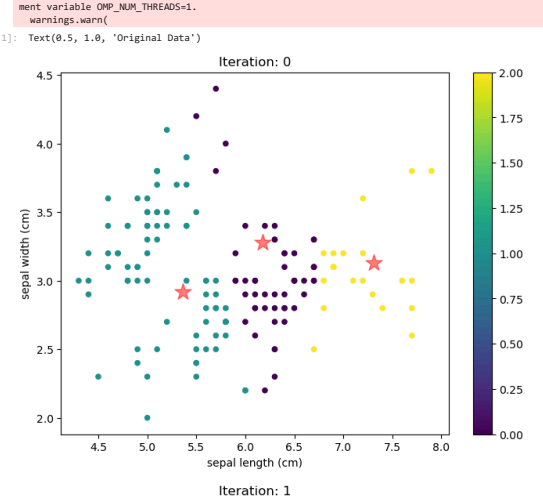
- Choose the number of k and a distance metric.
- Find the k nearest neighbors of the sample that we want to classify.
- Assign the class label by majority vote.
- Update the centroids of each class.
- Repeat the steps above until convergence.

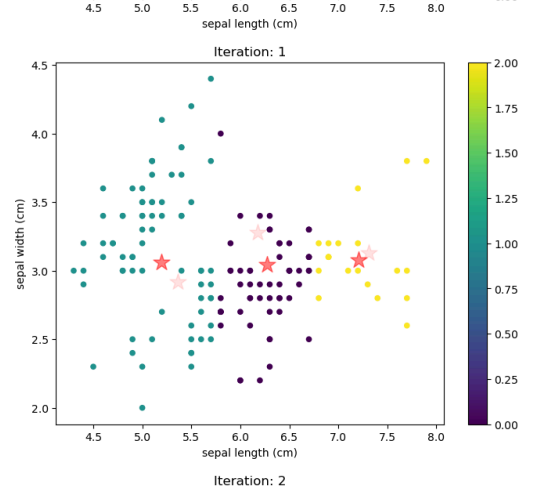
Note that because this algorithm takes distance into account, it is important that the features (columns) are on the same scale. For the iris dataset, they are but for other datasets, you may need to scale the features.

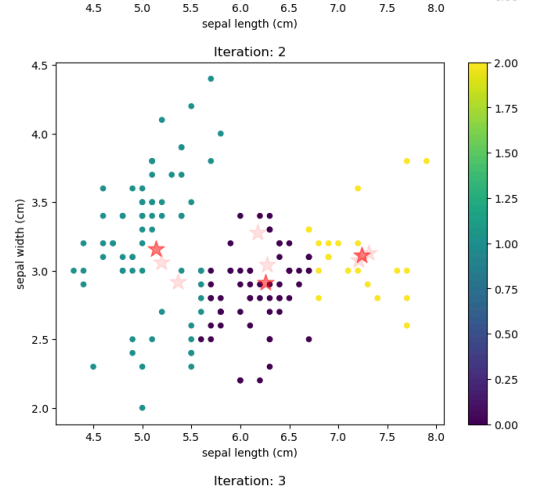
```
In [1]: from sklearn.cluster import KMeans
        import sklearn.datasets
        import matplotlib.pyplot as plt

        dataset = sklearn.datasets.load_iris(as_frame=True)

        X = (dataset
             ['data']
             .loc[:, ['sepal length (cm)', 'sepal width (cm)']]
        )
        y = dataset['target']

        # demonstration of k-means clustering with iris dataset
        # keep list of all centroids
        centroids = []

        # loop over a few iterations and plot the results
        for i in range(10):
            model = KMeans(n_clusters=3, init='random', n_init=1,
                           max_iter=i+1, random_state=42)
            model.fit(X)
            label = model.predict(X)
            # plot the input data color by cluster
            fig, ax = plt.subplots(figsize=(8, 6))
            X.plot.scatter(x='sepal length (cm)', y='sepal width (cm)', c=label, cmap='viridis', ax=ax)
            # plot the centers
            centers = model.cluster_centers_
            ax.scatter(centers[:, 0], centers[:, 1], marker='*', s=250, color='r', alpha=.5)
            ax.set_title('Iteration: ' + str(i))
            # plot previous centroids with reduced alpha value
            if i > 0:
                for centroid in centroids:
                    ax.scatter(centroid[:, 0], centroid[:, 1], marker='*', s=250, color='r', alpha=.1)
            # save the current centroids
            centroids.append(model.cluster_centers_)


          # plot the original data color by target
        fig, ax = plt.subplots(figsize=(8, 6))
        X.plot.scatter(x='sepal length (cm)', y='sepal width (cm)', c=y, cmap='viridis', ax=ax)
        ax.set_title('Original Data')
```

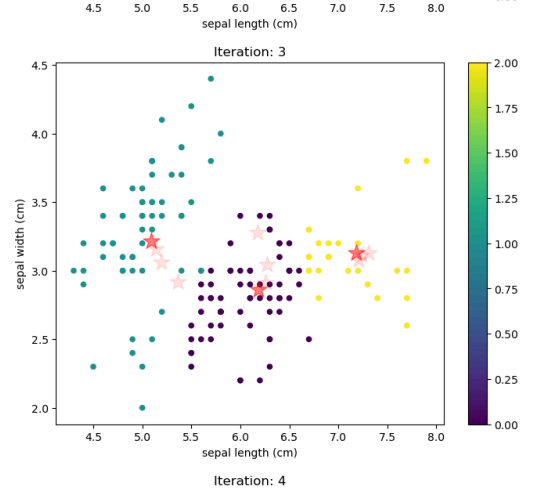
```
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

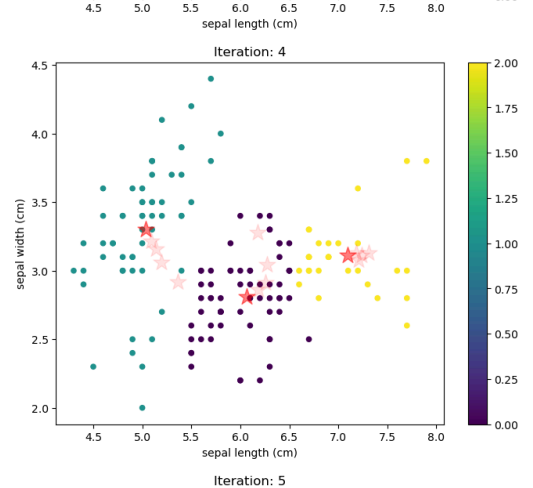
```
Out[1]: Text(0.5, 1.0, 'Original Data')
```

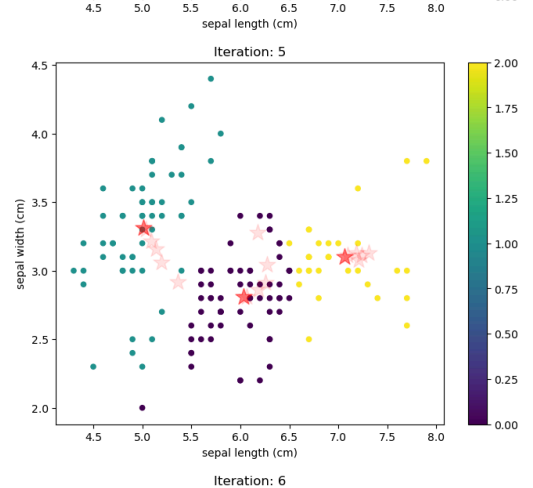

Iteration: 0

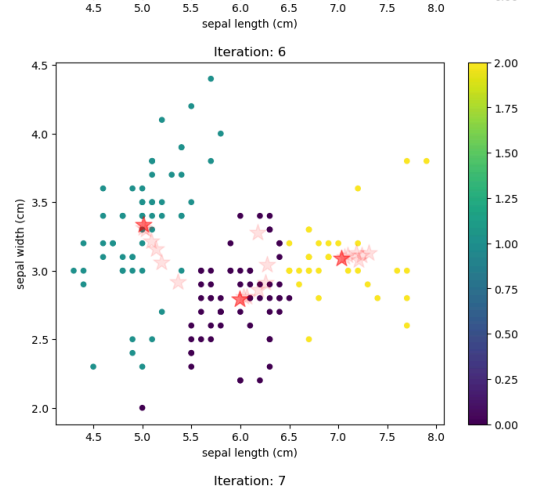

Iteration: 1

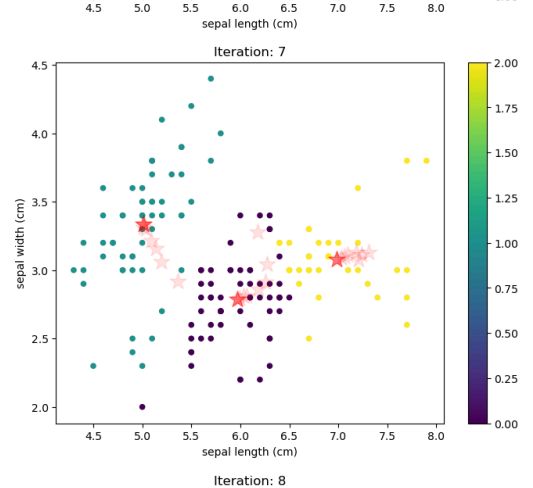

Iteration: 2

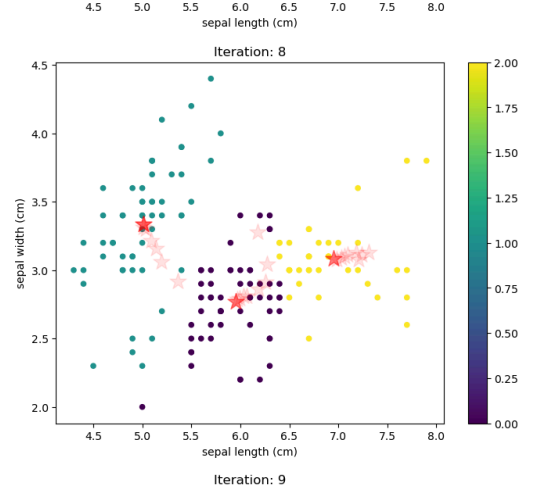

Iteration: 3

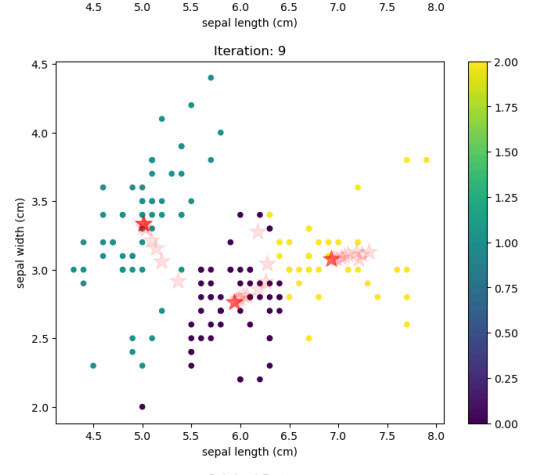

Iteration: 4

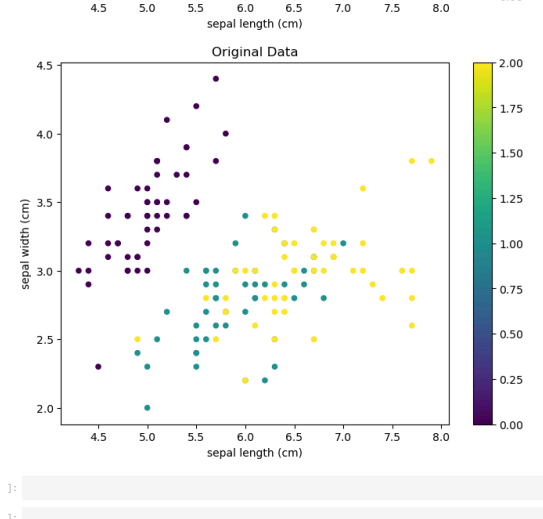

Iteration: 5


Iteration: 6


Iteration: 7


Iteration: 8


Iteration: 9


Original Data

```
In [ ]:
In [ ]:
In [ ]:
```

K Evaluation for K-means

We specify the K value for K-means clustering. However, we do not know the best K value for a given dataset.

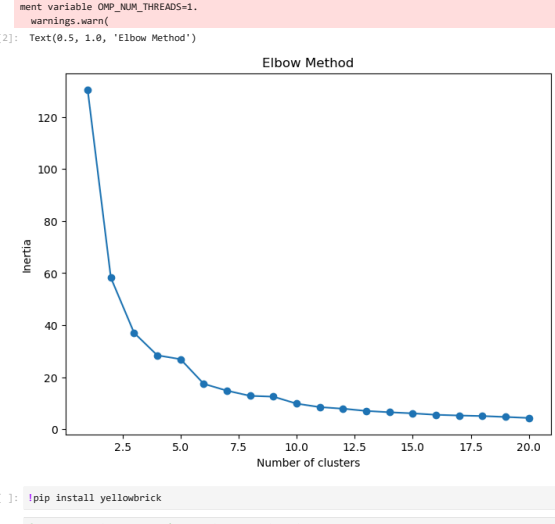
There are a few methods to evaluate the K value for K-means clustering:

- Elbow method - Track the "inertia" of the model as the K value increases. The inertia is the sum of squared distances of samples to their closest cluster center.
- Silhouette coefficient - The silhouette coefficient is a measure of how similar an object is to its own cluster compared to other clusters. The silhouette coefficient ranges from -1 to 1.

```
In [2]: # Run elbow method to find optimal number of clusters

        inertias = []
        max_clusters = 20
        for i in range(max_clusters):
            km = KMeans(n_clusters=i+1, n_init='auto',
                        max_iter=300, random_state=42)
            km.fit(X)
            inertias.append(km.inertia_)

        # plot the results
        fig, ax = plt.subplots(figsize=(8, 6))
        ax.plot(range(1, max_clusters+1), inertias, marker='o')
        ax.set_xlabel('Number of clusters')
        ax.set_ylabel('Inertia')
        ax.set_title('Elbow Method')
```

```
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
C:\Users\sahil\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1425: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environ
ment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

```
Out[2]: Text(0.5, 1.0, 'Elbow Method')
```


Elbow Method

```
In [ ]: !pip install yellowbrick
```

```
In [3]: from yellowbrick.cluster import SilhouetteVisualizer
        SilhouetteVisualizer?
```

```
Init signature:
SilhouetteVisualizer(
    estimator,
    ax=None,
    colors=None,
    is_fitted='auto',
    **kwargs,
)
Docstring:
The Silhouette Visualizer displays the silhouette coefficient for each
sample on a per-cluster basis, visually evaluating the density and
separation between clusters. The score is calculated by averaging the
silhouette coefficient for each sample, computed as the difference
between the average intra-cluster distance and the mean nearest-cluster
distance for each sample, normalized by the maximum value. This produces a
score between -1 and +1, where scores near +1 indicate high separation
and scores near -1 indicate that the samples may have been assigned to
the wrong cluster.

In SilhouetteVisualizer plots, clusters with higher scores have wider
silhouettes, but clusters that are less cohesive will fall short of the
average score across all clusters, which is plotted as a vertical dotted
red line.

This is particularly useful for determining cluster imbalance, or for
selecting a value for K by comparing multiple visualizers.

Parameters
----------
estimator : a Scikit-Learn clusterer
    Should be an instance of a centroidal clustering algorithm (``KMeans``
    or ``MiniBatchKMeans``). If the estimator is not fitted, it is fit when
    the visualizer is fitted, unless otherwise specified by ``is_fitted``.

ax : matplotlib Axes, default: None
    The axes to plot the figure on. If None is passed in the current axes
    will be used (or generated if required).

colors : iterable or string, default: None
    A collection of colors to use for each cluster group. If there are
    fewer colors than cluster groups, colors will repeat. May also be a
    Yellowbrick or matplotlib colormap string.

is_fitted : bool or str, default='auto'
    Specify if the wrapped estimator is already fitted. If False, the
    estimator will be fit when the visualizer is fit, otherwise, the
    estimator will not be modified. If 'auto' (default), a helper method
    will check if the estimator is fitted before fitting it again.

kwargs : dict
    Keyword arguments that are passed to the base class and may influence
    the visualization as defined in other Visualizers.

Attributes
----------
silhouette_score_ : float
    Mean Silhouette Coefficient for all samples. Computed via scikit-learn
    `sklearn.metrics.silhouette_score`.

silhouette_samples_ : array, shape = [n_samples]
    Silhouette Coefficient for each samples. Computed via scikit-learn
    `sklearn.metrics.silhouette_samples`.

n_samples_ : integer
    Number of total samples in the dataset (X.shape[0])

n_clusters_ : integer
    Number of clusters (e.g. n_clusters or k value) passed to internal
    scikit-learn model.

y_tick_pos_ : array of shape (n_clusters,)
    The computed center positions of each cluster on the y-axis

Examples
--------

>>> from yellowbrick.cluster import SilhouetteVisualizer
>>> from sklearn.cluster import KMeans
>>> model = SilhouetteVisualizer(KMeans(10))
>>> model.fit(X)
>>> model.show()
File:           c:\users\sahil\anaconda3\lib\site-packages\yellowbrick\cluster\silhouette.py
Type:           type
Subclasses:
```

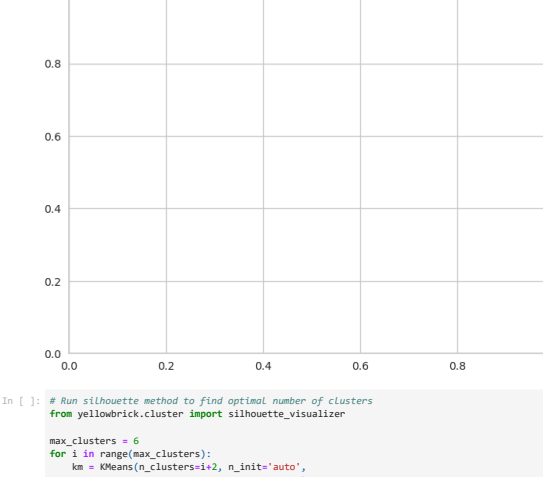
```
In [4]: # list matplotlib fonts available
        import matplotlib.font_manager
        matplotlib.font_manager.findSystemFonts(fontpaths=None, fontext='ttf')
```

```
Out[4]: ['C:\\Windows\\Fonts\\phagspa.ttf',
         'C:\\Windows\\Fonts\\comicbd.ttf',
         'C:\\Windows\\Fonts\\seguisbi.ttf',
         'C:\\Windows\\Fonts\\gadugib.ttf',
         'C:\\Windows\\Fonts\\msjhbd.ttc',
         'C:\\Windows\\Fonts\\consolai.ttf',
         'C:\\Windows\\Fonts\\mmrtextb.ttf',
         'C:\\Windows\\Fonts\\courbd.ttf',
         'C:\\Windows\\Fonts\\taileb.ttf',
         'C:\\Windows\\Fonts\\palab.ttf',
         'C:\\Windows\\Fonts\\DUBAI-LIGHT.TTF',
         'C:\\Windows\\Fonts\\trebucit.ttf',
         'C:\\Windows\\Fonts\\REFSAN.TTF',
         'C:\\Windows\\Fonts\\ANTQUAB.TTF',
         'C:\\Windows\\Fonts\\consolab.ttf',
         'C:\\Windows\\Fonts\\corbelli.ttf',
         'C:\\Windows\\Fonts\\CascadiaMono.ttf',
         'C:\\Windows\\Fonts\\calibriz.ttf',
         'C:\\Windows\\Fonts\\SimsunExtG.ttf',
         'C:\\Windows\\Fonts\\georgiaz.ttf',
         'C:\\Windows\\Fonts\\MTCORSVA.TTF',
         'C:\\Windows\\Fonts\\BOOKOS.TTF',
         'C:\\Windows\\Fonts\\corbelb.ttf',
         'C:\\Windows\\Fonts\\segoeuil.ttf',
         'C:\\Windows\\Fonts\\ARIALNI.TTF',
         'C:\\Windows\\Fonts\\msjhl.ttc',
         'C:\\Windows\\Fonts\\msyhbd.ttc',
         'C:\\Windows\\Fonts\\palai.ttf',
         'C:\\Windows\\Fonts\\calibrib.ttf',
         'C:\\Windows\\Fonts\\segoepr.ttf',
         'C:\\Windows\\Fonts\\comicz.ttf',
         'C:\\Windows\\Fonts\\segmdl2.ttf',
         'C:\\Windows\\Fonts\\cambriab.ttf',
         'C:\\Windows\\Fonts\\l_10646.ttf',
         'C:\\Windows\\Fonts\\segoescb.ttf',
         'C:\\Windows\\Fonts\\msjh.ttc',
         'C:\\Windows\\Fonts\\LEELAWDB.TTF',
         'C:\\Windows\\Fonts\\Candarai.ttf',
         'C:\\Windows\\Fonts\\BKANT.TTF',
         'C:\\Windows\\Fonts\\constan.ttf',
         'C:\\Windows\\Fonts\\cour.ttf',
         'C:\\Windows\\Fonts\\malgunsl.ttf',
         'C:\\Windows\\Fonts\\corbell.ttf',
         'C:\\Windows\\Fonts\\GARA.TTF',
         'C:\\Windows\\Fonts\\seguili.ttf',
         'C:\\Windows\\Fonts\\taile.ttf',
         'C:\\Windows\\Fonts\\msyh.ttc',
         'C:\\Windows\\Fonts\\webdings.ttf',
         'C:\\Windows\\Fonts\\verdana.ttf',
         'C:\\Windows\\Fonts\\seguisb.ttf',
         'C:\\Windows\\Fonts\\monbaiti.ttf',
         'C:\\Windows\\Fonts\\ANTQUABI.TTF',
         'C:\\Windows\\Fonts\\msyhl.ttc',
         'C:\\Windows\\Fonts\\simsunb.ttf',
         'C:\\Windows\\Fonts\\corbelz.ttf',
         'C:\\Windows\\Fonts\\segoeuiz.ttf',
         'C:\\Windows\\Fonts\\Candarali.ttf',
         'C:\\Windows\\Fonts\\CENTURY.TTF',
         'C:\\Windows\\Fonts\\georgia.ttf',
         'C:\\Windows\\Fonts\\segoeuib.ttf',
         'C:\\Windows\\Fonts\\ebrima.ttf',
         'C:\\Windows\\Fonts\\verdanab.ttf',
         'C:\\Windows\\Fonts\\seguisym.ttf',
         'C:\\Windows\\Fonts\\msyi.ttf',
         'C:\\Windows\\Fonts\\corbeli.ttf',
         'C:\\Windows\\Fonts\\MTEXTRA.TTF',
         'C:\\Windows\\Fonts\\comici.ttf',
         'C:\\Windows\\Fonts\\framdit.ttf',
         'C:\\Windows\\Fonts\\segoeuisl.ttf',
         'C:\\Windows\\Fonts\\Candara.ttf',
         'C:\\Windows\\Fonts\\sylfaen.ttf',
         'C:\\Windows\\Fonts\\ntailu.ttf',
         'C:\\Windows\\Fonts\\ARIALN.TTF',
         'C:\\Windows\\Fonts\\segoesc.ttf',
         'C:\\Windows\\Fonts\\pala.ttf',
         'C:\\Windows\\Fonts\\himalaya.ttf',
         'C:\\Windows\\Fonts\\CascadiaCode.ttf',
         'C:\\Windows\\Fonts\\LeelaUIb.ttf',
         'C:\\Windows\\Fonts\\GARABD.TTF',
         'C:\\Windows\\Fonts\\impact.ttf',
         'C:\\Windows\\Fonts\\WINGDNG3.TTF',
         'C:\\Windows\\Fonts\\calibril.ttf',
         'C:\\Windows\\Fonts\\LeelawUI.ttf',
         'C:\\Windows\\Fonts\\ebrimabd.ttf',
         'C:\\Windows\\Fonts\\ARIALNB.TTF',
         'C:\\Windows\\Fonts\\Candarab.ttf',
         'C:\\Windows\\Fonts\\georgiai.ttf',
         'C:\\Windows\\Fonts\\YuGothB.ttc',
         'C:\\Windows\\Fonts\\REFSPCL.TTF',
         'C:\\Windows\\Fonts\\courbi.ttf',
         'C:\\Windows\\Fonts\\DUBAI-REGULAR.TTF',
         'C:\\Windows\\Fonts\\WINGDNG2.TTF',
         'C:\\Windows\\Fonts\\timesbi.ttf',
         'C:\\Windows\\Fonts\\tahoma.ttf',
         'C:\\Windows\\Fonts\\calibrii.ttf',
         'C:\\Windows\\Fonts\\seguisli.ttf',
         'C:\\Windows\\Fonts\\arialbd.ttf',
         'C:\\Windows\\Fonts\\seguibl.ttf',
         'C:\\Windows\\Fonts\\palabi.ttf',
         'C:\\Windows\\Fonts\\GARAIT.TTF',
         'C:\\Windows\\Fonts\\framd.ttf',
         'C:\\Windows\\Fonts\\trebuc.ttf',
         'C:\\Windows\\Fonts\\georgiab.ttf',
         'C:\\Windows\\Fonts\\GOTHICI.TTF',
         'C:\\Windows\\Fonts\\arial.ttf',
         'C:\\Windows\\Fonts\\LEELAWAD.TTF',
         'C:\\Windows\\Fonts\\mmrtext.ttf',
         'C:\\Windows\\Fonts\\tahomabd.ttf',
         'C:\\Windows\\Fonts\\arialbi.ttf',
         'C:\\Windows\\Fonts\\SansSerifCollection.ttf',
         'C:\\Windows\\Fonts\\consola.ttf',
         'C:\\Windows\\Fonts\\segoeui.ttf',
         'C:\\Windows\\Fonts\\SegUIVar.ttf',
         'C:\\Windows\\Fonts\\DUBAI-BOLD.TTF',
         'C:\\Windows\\Fonts\\GOTHICBI.TTF',
         'C:\\Windows\\Fonts\\segoeuii.ttf',
         'C:\\Windows\\Fonts\\BOOKOSI.TTF',
         'C:\\Windows\\Fonts\\DUBAI-MEDIUM.TTF',
         'C:\\Windows\\Fonts\\javatext.ttf',
         'C:\\Windows\\Fonts\\cambria.ttc',
         'C:\\Windows\\Fonts\\simsun.ttc',
         'C:\\Windows\\Fonts\\BSSYM7.TTF',
         'C:\\Windows\\Fonts\\ariali.ttf',
         'C:\\Windows\\Fonts\\segoeprb.ttf',
         'C:\\Windows\\Fonts\\verdanai.ttf',
         'C:\\Windows\\Fonts\\constani.ttf',
         'C:\\Windows\\Fonts\\timesbd.ttf',
         'C:\\Windows\\Fonts\\Nirmala.ttc',
         'C:\\Windows\\Fonts\\lucon.ttf',
         'C:\\Windows\\Fonts\\YuGothR.ttc',
         'C:\\Windows\\Fonts\\YuGothM.ttc',
         'C:\\Windows\\Fonts\\gadugi.ttf',
         'C:\\Windows\\Fonts\\consolaz.ttf',
         'C:\\Windows\\Fonts\\seguihis.ttf',
         'C:\\Windows\\Fonts\\Gabriola.ttf',
         'C:\\Windows\\Fonts\\micross.ttf',
         'C:\\Windows\\Fonts\\seguiemj.ttf',
         'C:\\Windows\\Fonts\\Inkfree.ttf',
         'C:\\Windows\\Fonts\\ARIALNBI.TTF',
         'C:\\Windows\\Fonts\\SitkaVF-Italic.ttf',
         'C:\\Windows\\Fonts\\trebucbi.ttf',
         'C:\\Windows\\Fonts\\couri.ttf',
         'C:\\Windows\\Fonts\\ariblk.ttf',
         'C:\\Windows\\Fonts\\malgun.ttf',
         'C:\\Windows\\Fonts\\BOOKOSB.TTF',
         'C:\\Windows\\Fonts\\MSUIGHUB.TTF',
         'C:\\Windows\\Fonts\\corbel.ttf',
         'C:\\Windows\\Fonts\\calibrili.ttf',
         'C:\\Windows\\Fonts\\GOTHIC.TTF',
         'C:\\Windows\\Fonts\\ntailub.ttf',
         'C:\\Windows\\Fonts\\constanb.ttf',
         'C:\\Windows\\Fonts\\mmrtextb.ttf',
         'C:\\Windows\\Fonts\\verdanaz.ttf',
         'C:\\Windows\\Fonts\\times.ttf',
         'C:\\Windows\\Fonts\\timesi.ttf',
         'C:\\Windows\\Fonts\\bahnschrift.ttf',
         'C:\\Windows\\Fonts\\mvboli.ttf',
         'C:\\Windows\\Fonts\\cambriaz.ttf',
         'C:\\Windows\\Fonts\\phagspab.ttf',
         'C:\\Windows\\Fonts\\Candaraz.ttf',
         'C:\\Windows\\Fonts\\wingding.ttf',
         'C:\\Windows\\Fonts\\BOOKOSBI.TTF',
         'C:\\Windows\\Fonts\\SitkaVF.ttf',
         'C:\\Windows\\Fonts\\YuGothL.ttc',
         'C:\\Windows\\Fonts\\seguibli.ttf',
         'C:\\Windows\\Fonts\\mingliub.ttc',
         'C:\\Windows\\Fonts\\Candarail.ttf',
         'C:\\Windows\\Fonts\\calibri.ttf',
         'C:\\Windows\\Fonts\\MSUIGHUR.TTF',
         'C:\\Windows\\Fonts\\ANTQUAI.TTF',
         'C:\\Windows\\Fonts\\comic.ttf',
         'C:\\Windows\\Fonts\\SegoeIcons.ttf',
         'C:\\Windows\\Fonts\\GOTHICB.TTF',
         'C:\\Windows\\Fonts\\symbol.ttf',
         'C:\\Windows\\Fonts\\LeelUIsl.ttf',
         'C:\\Windows\\Fonts\\cambriai.ttf',
         'C:\\Windows\\Fonts\\malgunbd.ttf',
         'C:\\Windows\\Fonts\\trebucbd.ttf',
         'C:\\Windows\\Fonts\\constanz.ttf']
```

```
In [5]: # if on linux, set matplotlib font to DejaVuSans
        # get rid of warnings in next cell
        import matplotlib
        matplotlib.rcParams['font.family'] = 'DejaVu Sans'
```

```
In [7]: # Run silhouette method to find optimal number of clusters
        from yellowbrick.cluster import silhouette_visualizer

        max_clusters = 6
        for i in range(max_clusters):
            km = KMeans(n_clusters=i+2, n_init='auto',
                        max_iter=300, random_state=42)

            fig, ax = plt.subplots(figsize=(8, 6))
            # setting show=False so we can set xlim to same value for all plots
            viz = silhouette_visualizer(km, X, colors='yellowbrick', ax=ax, show=False)
            ax.set_xlim([-0.1, .8])
```

```
YellowbrickTypeError                      Traceback (most recent call last)
Cell In[7], line 11
      9 fig, ax = plt.subplots(figsize=(8, 6))
     10 # setting show=False so we can set xlim to same value for all plots
---> 11 viz = silhouette_visualizer(km, X, colors='yellowbrick', ax=ax, show=False)
     12 ax.set_xlim([-0.1, .8])

File ~\anaconda3\Lib\site-packages\yellowbrick\cluster\silhouette.py:324, in silhouette_visualizer(estimator, X, y, ax, colors, is_fitted, show, **kwargs)
    267 def silhouette_visualizer(
    268     estimator, X, y=None, ax=None, colors=None, is_fitted="auto", show=True, **kwargs
    269 ):
    270     """Quick Method:
    271     The Silhouette Visualizer displays the silhouette coefficient for each
    272     sample on a per-cluster basis, visually evaluating the density and
  (...)
    321         The silhouette visualizer, fitted and finalized.
    322     """
--> 324     oz = SilhouetteVisualizer(
    325         estimator, ax=ax, colors=colors, is_fitted=is_fitted, **kwargs
    326     )
    327     oz.fit(X, y)
    329     if show:

File ~\anaconda3\Lib\site-packages\yellowbrick\cluster\silhouette.py:118, in SilhouetteVisualizer.__init__(self, estimator, ax, colors, is_fitted, **kwargs)
    115 def __init__(self, estimator, ax=None, colors=None, is_fitted="auto", **kwargs):
    116
    117     # Initialize the visualizer bases
--> 118     super(SilhouetteVisualizer, self).__init__(estimator, ax=ax, **kwargs)
    120     # Visual Properties
    121     # Use colors if it is given, otherwise attempt to use colormap which
    122     # which will override colors. If neither is found, default to None.
    123     # The colormap may yet still be found in resolve_colors
    124     self.colors = colors

File ~\anaconda3\Lib\site-packages\yellowbrick\cluster\base.py:45, in ClusteringScoreVisualizer.__init__(self, estimator, ax, fig, force_model, **kwargs)
     43 def __init__(self, estimator, ax=None, fig=None, force_model=False, **kwargs):
     44     if not force_model and not isclusterer(estimator):
---> 45         raise YellowbrickTypeError(
     46             "The supplied model is not a clustering estimator; try a "
     47             "classifier or regression score visualizer instead!"
     48         )
     49     self.force_model = force_model
     50     super(ClusteringScoreVisualizer, self).__init__(
     51         estimator, ax=ax, fig=fig, **kwargs
     52     )

YellowbrickTypeError: The supplied model is not a clustering estimator; try a classifier or regression score visualizer instead!
```



```
In [ ]: # Run silhouette method to find optimal number of clusters
        from yellowbrick.cluster import silhouette_visualizer

        max_clusters = 6
        for i in range(max_clusters):
            km = KMeans(n_clusters=i+2, n_init='auto',
                        max_iter=300, random_state=42)

            fig, ax = plt.subplots(figsize=(8, 6))
            # setting show=False so we can set xlim to same value for all plots
            viz = silhouette_visualizer(km, X, colors='yellowbrick', ax=ax, show=False)
            ax.set_xlim([-0.1, .8])
```

```
In [ ]:
In [ ]:
```

Understanding Clustering Results

We can use the following methods to understand the clustering results:

- Create a surrogate model to predict the cluster label for a given sample.
- Summarize the cluster by the mean of each feature.
- Visualize the clustering results in 2D or 3D.

```
In [ ]: !pip install datasets
```

```
In [8]: # going to cluster electricity data from Australia
        # https://www.openml.org/search?type=data&sort=runs&id=151&status=active
        from datasets import load_dataset
        electricity = load_dataset('inria-soda/tabular-benchmark', data_files='clf_num/electricity.csv')
```

```
In [9]: electricity
```

```
Out[9]: DatasetDict({
            train: Dataset({
                features: ['date', 'period', 'nswprice', 'nswdemand', 'vicprice', 'vicdemand', 'transfer', 'class'],
                num_rows: 38474
            })
        })
```

```
In [10]: print(dir(electricity['train']))
```

```
['_TF_DATASET_REFS', '__class__', '__del__', '__delattr__', '__dict__', '__dir__', '__doc__', '__enter__', '__eq__', '__exit__', '__firstlineno__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__setstate__', '__sizeof__', '__static_attributes__', '__str__', '__subclasshook__', '__weakref__', '_build_local_temp_path', '_check_index_is_initialized', '_data', '_estimate_nbytes', '_fast_select_column', '_fingerprint', '_format_columns', '_format_kwargs', '_format_type', '_generate_tables_from_shards', '_get_cache_file_path', '_get_output_signature', '_getitem', '_indexes', '_indices', '_info', '_map_single', '_new_dataset_with_indices', '_output_all_columns', '_push_parquet_shards_to_hub', '_push_parquet_shards_to_hub_single', '_save_to_disk_single', '_select_contiguous', '_select_with_indices_mapping', '_split', 'add_column', 'add_elasticsearch_index', 'add_faiss_index', 'add_faiss_index_from_external_arrays', 'add_item', 'align_labels_with_mapping', 'batch', 'builder_name', 'cache_files', 'cast', 'cast_column', 'citation', 'class_encode_column', 'cleanup_cache_files', 'column_names', 'config_name', 'data', 'dataset_size', 'description', 'download_checksums', 'download_size', 'drop_index', 'features', 'filter', 'flatten', 'flatten_indices', 'format', 'formatted_as', 'from_buffer', 'from_csv', 'from_dict', 'from_file', 'from_generator', 'from_json', 'from_list', 'from_pandas', 'from_parquet', 'from_polars', 'from_spark', 'from_sql', 'from_text', 'get_index', 'get_nearest_examples', 'get_nearest_examples_batch', 'homepage', 'info', 'is_index_initialized', 'iter', 'license', 'list_indexes', 'load_elasticsearch_index', 'load_faiss_index', 'load_from_disk', 'map', 'num_columns', 'num_rows', 'push_to_hub', 'remove_columns', 'rename_column', 'rename_columns', 'repeat', 'reset_format', 'save_faiss_index', 'save_to_disk', 'search', 'search_batch', 'select', 'select_columns', 'set_format', 'set_transform', 'shape', 'shard', 'shuffle', 'size_in_bytes', 'skip', 'sort', 'split', 'supervised_keys', 'take', 'to_csv', 'to_dict', 'to_iterable_dataset', 'to_json', 'to_list', 'to_pandas', 'to_parquet', 'to_polars', 'to_sql', 'to_tf_dataset', 'train_test_split', 'unique', 'version', 'with_format', 'with_transform']
```

```
In [11]: elec = electricity['train'].to_pandas()
         elec
```

Out[11]:

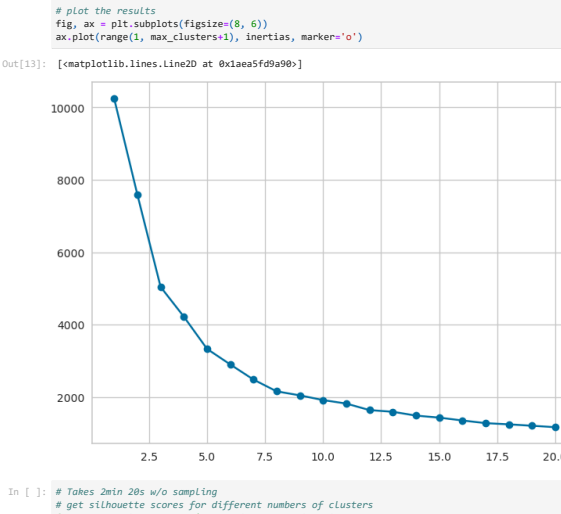
	date	period	nswprice	nswdemand	vicprice	vicdemand	transfer	class
0	0.898987	0.957447	0.068632	0.568283	0.004456	0.456499	0.644737	DOWN
1	0.867616	0.234043	0.033716	0.337102	0.003072	0.329622	0.846930	DOWN
2	0.009159	0.255319	0.059175	0.185808	0.003467	0.422915	0.414912	DOWN
3	0.898887	0.531915	0.087577	0.539572	0.004936	0.637752	0.491667	DOWN
4	0.866280	0.085106	0.027021	0.165129	0.001271	0.265924	0.748246	DOWN
...	...	...	...	...	...	...	...	...
38469	0.915800	0.404255	0.077549	0.456263	0.005332	0.378560	0.356140	UP
38470	0.915800	0.425532	0.074397	0.444213	0.005110	0.377525	0.369737	UP
38471	0.915800	0.468085	0.072835	0.421398	0.005519	0.354480	0.380263	UP
38472	0.915800	0.829787	0.065426	0.353913	0.004508	0.319524	0.318737	UP
38473	0.915800	0.978723	0.066651	0.329366	0.004630	0.345417	0.206579	UP

38474 rows × 8 columns

```
In [12]: X = elec.drop(columns=['class'])
```

```
In [13]: # get inertias for different numbers of clusters
         inertias = []
         max_clusters = 20
         for i in range(max_clusters):
             km = KMeans(n_clusters=i+1, n_init='auto',
                         max_iter=300, random_state=42)
             km.fit(X)
             inertias.append(km.inertia_)

         # plot the results
         fig, ax = plt.subplots(figsize=(8, 6))
         ax.plot(range(1, max_clusters+1), inertias, marker='o')
```

```
Out[13]: [<matplotlib.lines.Line2D at 0x1aa4f9e48d0>]
```



```
In [ ]: # Takes 3min 28s w/o sampling
        # get silhouette scores for different numbers of clusters
        from yellowbrick.cluster import silhouette_visualizer

        max_clusters = 5
        for i in range(max_clusters):
            km = KMeans(n_clusters=i+2, n_init='auto',
                        max_iter=300, random_state=42)

            fig, ax = plt.subplots(figsize=(8, 6))
            # setting show=False so we can set xlim to same value for all plots
            viz = silhouette_visualizer(km, X.sample(1_000, random_state=42), colors='yellowbrick', ax=ax, show=False)
            ax.set_xlim([-0.1, .8])
```

```
In [ ]: # going to choose 5 clusters (if 3, 6, or 7 are all close, choose the simpler model)
        # summarize the results by group

        km = KMeans(n_clusters=5, n_init='auto',
                    max_iter=300, random_state=42)
        km.fit(X)
        label = km.predict(X)
        (elec
         .assign(cluster=label)
         .groupby('cluster')
         .agg('mean', numeric_only=True)
         .T
         .style
         .background_gradient(cmap='RdBu', axis='columns')
        )
```

```
In [ ]: # Summarize by surrogate model decision tree
        from sklearn.tree import DecisionTreeClassifier

        dt = DecisionTreeClassifier(max_depth=3, random_state=42)
        dt.fit(X, label)
        # plot the tree
        from sklearn.tree import plot_tree
        fig, ax = plt.subplots(figsize=(14, 8))
        # make string for class names
        class_names = [str(i) for i in range(0, 5)]
        _ = plot_tree(dt, ax=ax, feature_names=X.columns, class_names=class_names, filled=True, fontsize=10)
```

```
In [ ]:
In [ ]:
In [ ]:
```

Other Clustering Algorithms

- Hierarchical clustering - This clusters by distance. It first clusters the two closest points, then clusters the next closest point to the first cluster, and so on. It is not as efficient as k-means, but it does not require you to specify the number of clusters. It is also more robust to outliers than k-means.
- DBSCAN - This clusters by density. It finds the densest region of points and clusters them together. It is also more robust to outliers than k-means.

```
In [ ]:
In [ ]:
```

Challenge: K-means Clustering

With the Titanic dataset:

- drop the missing values
- drop the categorical features
- scale the features
- run K-means clustering
- use the elbow method to find the best K value

Solution: K-means Clustering

```
In [ ]: import pandas as pd
        url = 'https://github.com/mattharrison/datasets/raw/master/data/titanic3.xls'
        raw = pd.read_excel(url)
        raw
```

```
In [ ]: raw.isna().sum()
```

```
In [ ]: raw.dtypes
```

```
In [ ]: raw.columns
```

```
In [ ]: def tweak_titanic(df):
          return (df
                  .loc[:, ['pclass', 'survived', 'age', 'sibsp', 'parch',
                   'fare']]
                   .dropna()
          )

        tweak_titanic(raw)
```

```
In [ ]: # standardize the data
        from sklearn.preprocessing import StandardScaler
        # have sklearn output pandas
        from sklearn import set_config
        set_config(transform_output='pandas')
        scaler = StandardScaler()
        X = scaler.fit_transform(tweak_titanic(raw))
        X
```

```
In [ ]: inertias = []
        for i in range(20):
            km = KMeans(n_clusters=i+1, n_init=1,
                        max_iter=300, random_state=42)
            km.fit(X)
            inertias.append(km.inertia_)

        # plot the results
        fig, ax = plt.subplots(figsize=(8, 6))
        ax.plot(range(1, 21), inertias, marker='o')
```

```
In [ ]:
In [ ]:
In [ ]:
```

PCA

Intro to PCA

Principal component analysis (PCA) is a dimensionality reduction technique. It finds the linear combinations of the features that explain the most variance in the data.

```
In [ ]: from sklearn.cluster import KMeans
        import sklearn.datasets
        import matplotlib.pyplot as plt

        dataset = sklearn.datasets.load_iris(as_frame=True)

        X = (dataset
             ['data']
             .loc[:, ['sepal length (cm)', 'sepal width (cm)']]
        )
        y = dataset['target']
```

```
In [ ]: X
```

```
In [ ]: X_pca
```

```
In [ ]: # Run PCA on the first two components of the iris dataset
        from sklearn.decomposition import PCA
        pca = PCA(n_components=2)
        pca.fit(X)
        X_pca = pca.transform(X)

        # plot the results
        fig, ax = plt.subplots(figsize=(8, 6))
        ax.scatter(X_pca['pca0'], X_pca['pca1'], c=y, cmap='viridis')
        ax.set_xlabel('PC1')
        ax.set_ylabel('PC2')
        ax.set_title('PCA on Iris Data')
```

```
In [ ]: X
```

```
In [ ]: # Run PCA on all of the iris dataset
        pca = PCA()

        X_all = (dataset
             ['data']
             #.loc[:, ['sepal length (cm)', 'sepal width (cm)']]
        )
        pca.fit(X_all)
        X_pca = pca.transform(X_all)

        # Plot to first 2 components
        fig, ax = plt.subplots(figsize=(8, 6))
        ax.scatter(X_pca['pca0'], X_pca['pca1'], c=y, cmap='viridis')
        ax.set_xlabel('PC1')
        ax.set_ylabel('PC2')
        ax.set_title('PCA on Iris Data')
```

```
In [ ]:
In [ ]:
In [ ]:
```

Components of PCA

When we run PCA, we get back the following:

- Principal components (PCs): The PCs are the linear combinations of the features.
- Explained variance ratio: The explained variance ratio tells us how much variance is explained by each PC.
- Feature weights: The feature weights tell us how much each feature contributes to each PC.

```
In [ ]: # tell sklearn to output pandas dataframes
        sklearn.set_config(transform_output='pandas')

        pca.transform(X_all)
```

```
In [ ]: # I don't like the column names, so I'll rename them
        # change pca0 to PC1, pca1 to PC2, etc

        def rename_pc0_to_PC1(col):
            num = int(col[3:]) + 1
            return 'PC' + str(num)

        pca.transform(X_all).rename(columns=rename_pc0_to_PC1)
```

```
In [ ]: # Scree plot of explained variance
        fig, ax = plt.subplots(figsize=(8, 6))
        ax.plot(range(1, len(pca.explained_variance_ratio_)+1), pca.explained_variance_ratio_, marker='o')
        ax.set_xlabel('Principal Component')
        ax.set_ylabel('Explained Variance Ratio')
        ax.set_title('Scree Plot')
```

```
In [ ]: # The explained variance ratio is the percentage of variance explained by each of the selected components.
        # The first principal component explains 92.5% of the variance in the data, and
        # the second principal component explains 5.3% of the variance in the data.
        pca.explained_variance_ratio_
```

```
In [ ]: pca.explained_variance_ratio_.cumsum()
```

```
In [ ]:
In [ ]:
In [ ]:
In [ ]:
```

Weights

For every PC, we get a set of weights for each feature. The weights tell us how much each feature contributes to the PC.

```
In [ ]: # convert components to a dataframe
        import pandas as pd

        components = pd.DataFrame(pca.components_, columns=X_all.columns,
                                  index=['PC1', 'PC2', 'PC3', 'PC4'])

        components
```

```
In [ ]: # Centered data - for next cell's calculation
        X_all - X_all.mean()
```

```
In [ ]: # calculating PC1 by hand for first row - linear combination of
        # centered variables and the first component
        -.743333 * .3613 + 0.4426 * -0.0845 + -2.358 * 0.8566 + -0.9993 * 0.3582
```

```
In [ ]: pca.transform(X_all).rename(columns=rename_pc0_to_PC1)
```

```
In [ ]: # Manually calculating PCA with numpy
        import numpy as np
        nums = X_all - X_all.mean()
        vals, vecs = np.linalg.eig(nums.cov())
        idxs = pd.Series(vals).argsort()

        explained_variance = pd.Series(sorted(vals, reverse=True))

        def set_columns(df_):
            df_.columns = [f'PC{i+1}' for i in range(len(df_.columns))]
            return df_

        comps = (pd.DataFrame(vecs, index=nums.columns)
          .iloc[:, idxs[::-1]]
          .pipe(set_columns)
        )

        pcas = (nums.dot(comps))
        pcas
```

```
In [ ]:
```

Scatter Plot

```
In [ ]: # Use plotly to plot the first three components
        import plotly.express as px
        fig = px.scatter_3d(pcas, x='PC1', y='PC2', z='PC3', color=y)
        fig.show()
```

```
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
```

Other Dimensionality Reduction Techniques

- t-SNE - t-distributed stochastic neighbor embedding. Tries to preserve the local structure of the data.
- UMAP - Uniform Manifold Approximation and Projection. Tries to preserve the both global and structure of the data.
- Autoencoders

```
In [ ]: # run t-SNE on the iris dataset
        from sklearn.manifold import TSNE
        tsne = TSNE(n_components=3, random_state=42)
        X_tsne = tsne.fit_transform(X_all)
```

```
In [ ]: X_tsne
```

```
In [ ]: # plot results with plotly
        import plotly.express as px
        fig = px.scatter_3d(X_tsne, x='tsne0', y='tsne1', z='tsne2', color=y)
        fig.show()
```

```
In [ ]: !pip install umap-learn
```

```
In [ ]: # run UMAP
        import umap
        reducer = umap.UMAP(random_state=42, n_components=3)
        X_umap = pd.DataFrame(reducer.fit_transform(X_all), columns=['umap0', 'umap1', 'umap2'])
        X_umap
```

```
In [ ]: # plot results with plotly
        import plotly.express as px
        fig = px.scatter_3d(X_umap, x='umap0', y='umap1', z='umap2', color=y)
        fig.show()
```

```
In [ ]:
In [ ]:
In [ ]:
```

Challenge: PCA

Run PCA on the numeric columns of the Titanic data. Plot the result of the first three dimensions using plotly.

```
In [ ]:
```

Solution: PCA

```
In [ ]: tweak_titanic(raw)
```

```
In [ ]: # standardize the data
        from sklearn.preprocessing import StandardScaler
        scaler = StandardScaler()

        X = scaler.fit_transform(tweak_titanic(raw).drop(columns='survived'))
        X
```

```
In [ ]: # run pca on the data
        pca = PCA()

        X_pca = pca.fit_transform(X)
```

```
In [ ]: X_pca
```

```
In [ ]:
```

```
In [ ]: # plot the first 3 columns in plotly
        import plotly.express as px

        fig = px.scatter_3d(X_pca, x='pca0', y='pca1', z='pca2', color=raw.loc[X_pca.index]['survived'])
        fig.show()
```

```
In [ ]:
In [ ]:
In [ ]:
```

Linear Regression

```
In [ ]:
```

Linear Regression Algorithm

Linear regression calculates an intercept and slope (weights) for a line that minimizes the sum of squared errors between the line and the data points.

The formula for linear regression is as follows:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

where y is the target variable, β_0 is the intercept, β_1 to β_n are the weights, and x_1 to x_n are the features.

The algorithm is as follows:

- Initialize the weights.
- Calculate the predicted values.
- Calculate the error.
- Update the weights.
- Repeat the steps above until convergence.

```
In [ ]: # Load anscombe's quartet
        x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
        y1 = [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68]
        y2 = [9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74]
        y3 = [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73]
        x4 = [8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8]
        y4 = [6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89]
        anscombe = pd.DataFrame({'x': x, 'y1': y1, 'y2': y2, 'y3': y3, 'x4': x4, 'y4': y4})
                 )

        anscombe
```

```
In [ ]: # plot x y1
        fig, ax = plt.subplots(figsize=(3, 3))
        anscombe.plot.scatter(x='x', y='y1', ax=ax, color='k')
```

Let's run the algorithm on x and y1

Calculate the slope:

$$\beta_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Calculate the intercept:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

Model Equation:

$$y = \beta_0 + \beta_1 x$$

```
In [ ]: # slope

        x1 = anscombe['x']
        y1 = anscombe['y1']
        slope = ((x1 - x1.mean())*(y1 - y1.mean())).sum() / ((x1 - x1.mean())**2).sum()
        slope
```

```
In [ ]: # intercept

        intercept = y1.mean() - slope * x1.mean()
        intercept
```

```
In [ ]: # plot x y1
        fig, ax = plt.subplots(figsize=(3, 3))
        anscombe.plot.scatter(x='x', y='y1', ax=ax, color='k')
        # plot the line
        x1 = np.linspace(4, 14, 100)
        y1 = slope * x1 + intercept
        ax.plot(x1, y1, color='r')
```

```
In [ ]:
In [ ]:
In [ ]:
```

Examples with Scikit-learn

```
In [ ]: from sklearn.linear_model import LinearRegression

        x1 = anscombe[['x']]
        y1 = anscombe['y1']
        y2 = anscombe['y2']
        y3 = anscombe['y3']

        lr1 = LinearRegression()
        lr1.fit(x1, y1)
```

```
In [ ]: lr1.coef_
```

```
In [ ]: lr1.intercept_
```

```
In [ ]: lr2 = LinearRegression()
        lr2.fit(x1, y2)
        lr3 = LinearRegression()
        lr3.fit(x1, y3)
```

```
In [ ]: # plot 1, 2 and 3 in different colors
        fig, axs = plt.subplots(1, 3, figsize=(9, 3))
        anscombe.plot.scatter(x='x', y='y1', ax=axs[0], color='k')
        axs[0].plot(x1, lr1.predict(x1), color='#aaa')
        axs[0].set_ylim(3, 13)
        anscombe.plot.scatter(x='x', y='y2', ax=axs[1], color='b')
        axs[1].plot(x1, lr2.predict(x1), color='#55a')
        axs[1].set_ylim(3, 13)
        anscombe.plot.scatter(x='x', y='y3', ax=axs[2], color='g')
        axs[2].plot(x1, lr3.predict(x1), color='#5a5')
        axs[2].set_ylim(3, 13)
```

```
In [ ]:
In [ ]:
In [ ]:
```

Real world example with Aircraft Elevators

From website: This data set is also obtained from the task of controlling a F16 aircraft, although the target variable and attributes are different from the ailerons domain. In this case the goal variable is related to an action taken on the elevators of the aircraft.

```
In [ ]: # https://www.openml.org/search?type=data&sort=runs&id=216&status=active
        from datasets import load_dataset
        elevators = load_dataset('inria-soda/tabular-benchmark', data_files='reg_num/elevators.csv')
```

```
In [ ]: elev = elevators['train'].to_pandas()
        elev
```

```
In [ ]: X = elev.drop(columns=['Goal'])
        y = elev['Goal']

        lr_elev = LinearRegression()
        lr_elev.fit(X, y)
```

```
In [ ]: lr_elev.coef_
```

```
In [ ]: lr_elev.intercept_
```

```
In [ ]: pd.Series(lr_elev.coef_, index=X.columns).sort_values().plot.barh(figsize=(8, 6))
```

```
In [ ]: # score is R^2 - the proportion of variance explained by the model
        lr_elev.score(X, y)
```

```
In [ ]: from sklearn.metrics import mean_squared_error, mean_absolute_error
        mean_absolute_error(y, lr_elev.predict(X)), mean_squared_error(y, lr_elev.predict(X))
```

```
In [ ]: lr_elev.predict(X.iloc[[0]])
```

```
In [ ]: y.iloc[0]
```

```
In [ ]:
In [ ]:
In [ ]:
```

Assumptions of Linear Regression

- Linear relationship between the features and target variable
- No multicollinearity - no correlation between the features
- Homoscedasticity - the variance of the residuals is the same for all values of the target variable
- No outliers - the residuals are normally distributed

Also, generally you will want to scale the features before running linear regression.

```
In [ ]: # standardize the data
        from sklearn.preprocessing import StandardScaler
        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)
        X_scaled
```

```
In [ ]: X_scaled.describe()
```

```
In [ ]: lr_std = LinearRegression()
        lr_std.fit(X_scaled, y)
        lr_std.score(X_scaled, y)
```

```
In [ ]: pd.Series(lr_std.coef_, index=X.columns).sort_values().plot.barh(figsize=(8, 6))
```

```
In [ ]: !pip install xgboost
```

```
In [ ]: # try with XGBoost
        from xgboost import XGBRegressor
        xgb = XGBRegressor()
        xgb.fit(X, y)
        xgb.score(X, y)
```

```
In [ ]:
In [ ]:
```

Challenge: Linear Regression

Make a model to predict how much Titanic passengers paid for their tickets with Linear Regression. (Only use the numeric columns for the model.)

```
In [ ]:
```

Solution: Linear Regression

```
In [ ]: raw
```

```
In [ ]: # predict fare from numeric columns
        X = tweak_titanic(raw).drop(columns=['fare'])
        y = tweak_titanic(raw)['fare']
```

```
In [ ]: # make linear regression model
        lr = LinearRegression()
        lr.fit(X, y)
```

```
In [ ]: lr.score(X, y)
```

```
In [ ]: # make an xgboost model
        xgb = XGBRegressor()

        xgb.fit(X,y)
        xgb.score(X,y)
```

```
In [ ]:
In [ ]:
In [ ]:
```

Logistic Regression

Logistic Regression Algorithm

Even though logistic regression has "regression" in its name, it is used as a classification algorithm. It calculates the probability that a sample belongs to a class. Rather than fitting a line to the data, it fits an "S" shaped curve called the sigmoid function.

The formula for logistic regression is as follows:

$$y = \frac{1}{1 + e^{-z}}$$

where y is the probability that a sample belongs to a class and z is the linear combination of the features.

The algorithm is as follows:

- Initialize the weights.
- Calculate the predicted values.
- Calculate the error.
- Update the weights.
- Repeat the steps above until convergence.

```
In [ ]: # plot sigmoid function
        import numpy as np
        from matplotlib import pyplot as plt
        x = np.linspace(-10, 10, 100)
        y = 1 / (1 + np.exp(-x))
        fig, ax = plt.subplots(figsize=(8, 6))
        ax.plot(x, y)
```

```
In [ ]:
In [ ]:
```

Basic Example

```
In [ ]: import pandas as pd
        log_data = pd.DataFrame({'x': [-2, -2.5, -3.1, -1, -.5, 0, .5, .7, 1, 2, 3],
                                 'y': [0, 0, 0, 0, 1, 0, 1,1, 1, 1, 1]})

        log_data.plot.scatter(x='x', y='y')
```

```
In [ ]: from sklearn.linear_model import LogisticRegression
        log_r = LogisticRegression()
        log_r.fit(log_data[['x']], log_data['y'])
```

```
In [ ]: log_r.coef_
```

```
In [ ]: log_r.intercept_
```

```
In [ ]: # plot fitted sigmoid function on top of data
        x = np.linspace(-5, 4, 100)
        y = 1 / (1 + np.exp(-(log_r.coef_[0][0] * x + log_r.intercept_[0])))
        fig, ax = plt.subplots(figsize=(8, 6))
        ax.plot(x, y)
        log_data.plot.scatter(x='x', y='y', ax=ax)
        # annotate about .5
        ax.annotate('Predict 1(right of this)', xy=(-.31, .5), xytext=(1, .6), arrowprops={'arrowstyle': '->'})
```

```
In [ ]: log_r.predict([[-.3]])
```

```
In [ ]:
In [ ]:
In [ ]:
```

Real World Example with Eye movements

From the website:

The dataset consist of several assignments. Each assignment consists of a question followed by ten sentences (titles of news articles). One of the sentences is the correct answer to the question (C) and five of the sentences are irrelevant to the question (I). Four of the sentences are relevant to the question (R), but they do not answer it.

- Features are in columns, feature vectors in rows.
- Each assignment is a time sequence of 22-dimensional feature vectors.
- The first column is the line number, second the assignment number and the next 22 columns (3 to 24) are the different features. Columns 25 to 27 contain extra information about the example. The training data set contains the classification label in the 28th column: "0" for irrelevant, "1" for relevant and "2" for the correct answer.
- Each example (row) represents a single word. You are asked to return the classification of each read sentence.
- The 22 features provided are commonly used in psychological studies on eye movement. All of them are not necessarily relevant in this context.

The objective of the Challenge is to predict the classification labels (I, R, C).

```
In [ ]: # https://www.openml.org/d/1044
        from datasets import load_dataset
        eye = load_dataset('inria-soda/tabular-benchmark', data_files='clf_num/eye_movements.csv')
```

```
In [ ]: eye_df = eye['train'].to_pandas()
        eye_df
```

```
In [ ]: from sklearn.preprocessing import StandardScaler

        X = eye_df.drop(columns=['label'])
        y = eye_df['label']
        std = StandardScaler()
        X_scaled = std.fit_transform(X)
        eye_log = LogisticRegression()
        eye_log.fit(X_scaled, y)
        eye_log.score(X_scaled, y)
```

```
In [ ]: pd.Series(eye_log.coef_[0], index=X.columns).sort_values().plot.barh(figsize=(8, 6))
```

```
In [ ]:
In [ ]:
In [ ]:
```

Challenge: Logistic Regression

Create a logistic regression model to predict whether a Titanic passenger survives based on the numeric columns.

```
In [ ]:
In [ ]:
```

Solution: Logistic Regression

```
In [ ]: import pandas as pd
        url = 'https://github.com/mattharrison/datasets/raw/master/data/titanic3.xls'
        raw = pd.read_excel(url)
        raw
```

```
In [ ]: # predicted survived column with logistic regression
        from sklearn.linear_model import LogisticRegression

        titanic_X = (raw
         .loc[:, ['pclass', 'age', 'sibsp', 'parch', 'fare']]
         .dropna()
                  )

        # standardize the data
        from sklearn.preprocessing import StandardScaler
        from sklearn import set_config
        set_config(transform_output='pandas')

        std = StandardScaler()
        X = std.fit_transform(titanic_X)
        X
```

```
In [ ]: # load train_test_split
        from sklearn.model_selection import train_test_split
        # split the data
        y = raw.loc[X.index, 'survived']

        X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
        lr = LogisticRegression()
        lr.fit(X_train, y_train)
        lr.score(X_test, y_test)
```

```
In [ ]: # try an XGBoost model
        from xgboost import XGBClassifier

        xgb = XGBClassifier()
        xgb.fit(X_train, y_train)
        xgb.score(X_test, y_test)
```

```
In [ ]:
```

Decision Trees

Decision Tree Algorithm

Decision trees are a type of supervised learning algorithm that can be used for both classification and regression. They work by splitting the data into subsets based on the features. The goal is to split the data in a way that minimizes the entropy of the subsets.

```
In [ ]: ## Create "decision stump"
        ## fit tree regressor to anscombe's quartet limit to 1 level

        from sklearn.tree import DecisionTreeRegressor
        dt = DecisionTreeRegressor(max_depth=1)
        X = anscombe[['x']]
        y = anscombe['y1']
        dt.fit(X, y)
```

```
In [ ]: ## Plot the tree
        from sklearn.tree import plot_tree
        fig, ax = plt.subplots(figsize=(8, 6))
        _ = plot_tree(dt, ax=ax, feature_names=['x'], filled=True, fontsize=10)
```

```
In [ ]: ## Plot the data and predictions on the same plot
        import numpy as np
        fig, ax = plt.subplots(figsize=(8, 6))
        anscombe.plot.scatter(x='x', y='y1', ax=ax, color='k')
        # plot the line
        x1 = np.linspace(4, 14, 100)
        y1 = dt.predict(x1.reshape(-1, 1))
        ax.plot(x1, y1, color='r')
```

```
In [ ]: ## Now plot to two levels
        fig, ax = plt.subplots(figsize=(8, 6))
        anscombe.plot.scatter(x='x', y='y1', ax=ax, color='k')
        # plot the line
        dt2 = DecisionTreeRegressor(max_depth=2)
        dt2.fit(X, y)

        x1 = np.linspace(4, 14, 100)
        y1 = dt2.predict(x1.reshape(-1, 1))
        ax.plot(x1, y1, color='r')
```

```
In [ ]: ## Now plot unlimited levels
        fig, ax = plt.subplots(figsize=(8, 6))
        anscombe.plot.scatter(x='x', y='y1', ax=ax, color='k')
        # plot the line
        dt3 = DecisionTreeRegressor(max_depth=None)
        dt3.fit(X, y)

        x1 = np.linspace(4, 14, 100)
        y1 = dt3.predict(x1.reshape(-1, 1))
        ax.plot(x1, y1, color='r')
```

```
In [ ]:
In [ ]:
In [ ]:
```

Real World with Aircraft Elevators

```
In [ ]: X_elev = elev.drop(columns=['Goal'])
        y_elev = elev['Goal']
        dt_elev = DecisionTreeRegressor(max_depth=3)
        dt_elev.fit(X_elev, y_elev)
```

```
In [ ]: # plot the tree
        from sklearn.tree import plot_tree
        fig, ax = plt.subplots(figsize=(12, 8))
        _ = plot_tree(dt_elev, ax=ax, feature_names=X_elev.columns, filled=True, fontsize=10, precision=4)
```

```
In [ ]: dt_elev.score(X_elev, y_elev)
```

```
In [ ]: from sklearn.linear_model import LinearRegression
        lr_elev = LinearRegression()
        lr_elev.fit(X_elev, y_elev)
        lr_elev.score(X_elev, y_elev)
```

```
In [ ]: # loop over depths and plot the results
        scores = []
        for i in range(1, 20):
            dt = DecisionTreeRegressor(max_depth=i)
            dt.fit(X_elev, y_elev)
            scores.append(dt.score(X_elev, y_elev))

        pd.Series(scores).plot.line(figsize=(8, 6))
```

```
In [ ]: # split the data and plot results of train and test
        from sklearn.model_selection import train_test_split
        X_train, X_test, y_train, y_test = train_test_split(X_elev, y_elev, random_state=42)
        test_scores = []
        train_scores = []
        for i in range(1, 20):
            dt = DecisionTreeRegressor(max_depth=i)
            dt.fit(X_train, y_train)
            test_scores.append(dt.score(X_test, y_test))
            train_scores.append(dt.score(X_train, y_train))

        ax = pd.DataFrame({'train': train_scores, 'test': test_scores}).plot.line(figsize=(8, 6))

        # annotate overfitting at 10, .7
        ax.annotate('Overfitting after here', xy=(10, .7), xytext=(12, .5), arrowprops={'arrowstyle': '->'})

        # set title
        ax.set_title('Validation Curve for Decision Tree')
```

```
In [ ]: # Let's see if our model improves with a deeper tree
        dt_elev = DecisionTreeRegressor(max_depth=11)
        dt_elev.fit(X_train, y_train)
        dt_elev.score(X_test, y_test)
```

```
In [ ]: lr_elev = LinearRegression()
        lr_elev.fit(X_train, y_train)
        lr_elev.score(X_test, y_test)
```

```
In [ ]:
```

Random Forests and XGBoost

```
In [ ]: # create a random forest regressor
        from sklearn.ensemble import RandomForestRegressor
        rf = RandomForestRegressor(n_estimators=100, max_depth=3)
        rf.fit(X_train, y_train)
        rf.score(X_test, y_test)
```

```
In [ ]: # sweep over depths and plot results
        test_scores = []
        train_scores = []
        for i in range(1, 20):
            rf = RandomForestRegressor(n_estimators=100, max_depth=i)
            rf.fit(X_train, y_train)
            test_scores.append(rf.score(X_test, y_test))
            train_scores.append(rf.score(X_train, y_train))

        ax = pd.DataFrame({'train': train_scores, 'test': test_scores}).plot.line(figsize=(8, 6))
```

```
In [ ]: # create a random forest regressor
        from sklearn.ensemble import RandomForestRegressor
        rf = RandomForestRegressor(n_estimators=100, max_depth=15, random_state=42)
        rf.fit(X_train, y_train)
        rf.score(X_test, y_test)
```

```
In [ ]: # create an xgb regressor
        from xgboost import XGBRegressor
        xgb = XGBRegressor()
        xgb.fit(X_train, y_train)
        xgb.score(X_test, y_test)
```

```
In [ ]:
In [ ]:
In [ ]:
```

Challenge: Decision Trees

Create a decision tree to predict survival on the titanic. See if you can determine the optimal depth of the tree.

Solution: Decision Trees

```
In [ ]:
```

```
In [ ]: from sklearn.tree import DecisionTreeClassifier

        raw
```

```
In [ ]: titanic_X = (raw
                     .loc[:, ['pclass', 'age', 'sibsp', 'parch', 'fare']]
                     .dropna()
                     )

        titanic_X
        titanic_y = (raw
         ['survived']
         .loc[titanic_X.index]
         )
```

```
In [ ]: # split data
        X_train, X_test, y_train, y_test = train_test_split(titanic_X, titanic_y, random_state=42)
```

```
In [ ]: X_train
```

```
In [ ]: # sweep over depths and plot results
        test_scores = []
        train_scores = []
        for i in range(1, 20):
            dt = DecisionTreeClassifier(max_depth=i)
            dt.fit(X_train, y_train)
            test_scores.append(dt.score(X_test, y_test))
            train_scores.append(dt.score(X_train, y_train))

        ax = pd.DataFrame({'train': train_scores, 'test': test_scores}).plot.line(figsize=(8, 6))
```

```
In [ ]:
In [ ]:
```

Conclusion - Next Steps

- Practice, practice, practice! - I recommend using your own data to practice.
- Check out my Feature Engineering course.
- Check out my XGBoost course.